

ARCHITECTURE PDF EDITION

# Architecture Map

How the Ontology Machine is wired: surfaces, runtime flow, corpus layers, agents, and failure routing.

**Core rule:** owner surfaces define truth; the pipeline materializes evidence; agents work through controlled contracts.

# Contents

- 1. 2. Architecture Map** ..... 3
- 2. Top-Level Topology ..... 4
- 3. Ownership Map ..... 5
- 4. Mainline Ingestion Map ..... 6
- 5. Artifact Flow ..... 8
- 6. Corpus DB Layer Map ..... 9
- 7. Semantic Release Map ..... 11
- 8. Control Plane Map ..... 12
- 9. Agent Map ..... 14
- 10. HTTP Surface Map ..... 16
- 11. State Map ..... 17
- 12. Merge And Rebuild Map ..... 18
- 13. Failure Routing Map ..... 19
- 14. Change Orientation ..... 20
- 15. Implementation Anchors ..... 21
- 16. Reading Order ..... 22

Recommended first pass for new maintainers: read the topology, ownership map, mainline ingestion map, and failure routing map before touching code.

## 2. Architecture Map

This chapter maps how the Ontology Machine is wired. The System Overview explains what the Machine is. This map explains where the pieces live, how data moves between them, which runtime surface owns which decision, and where to start when something has to be changed.

The short version:

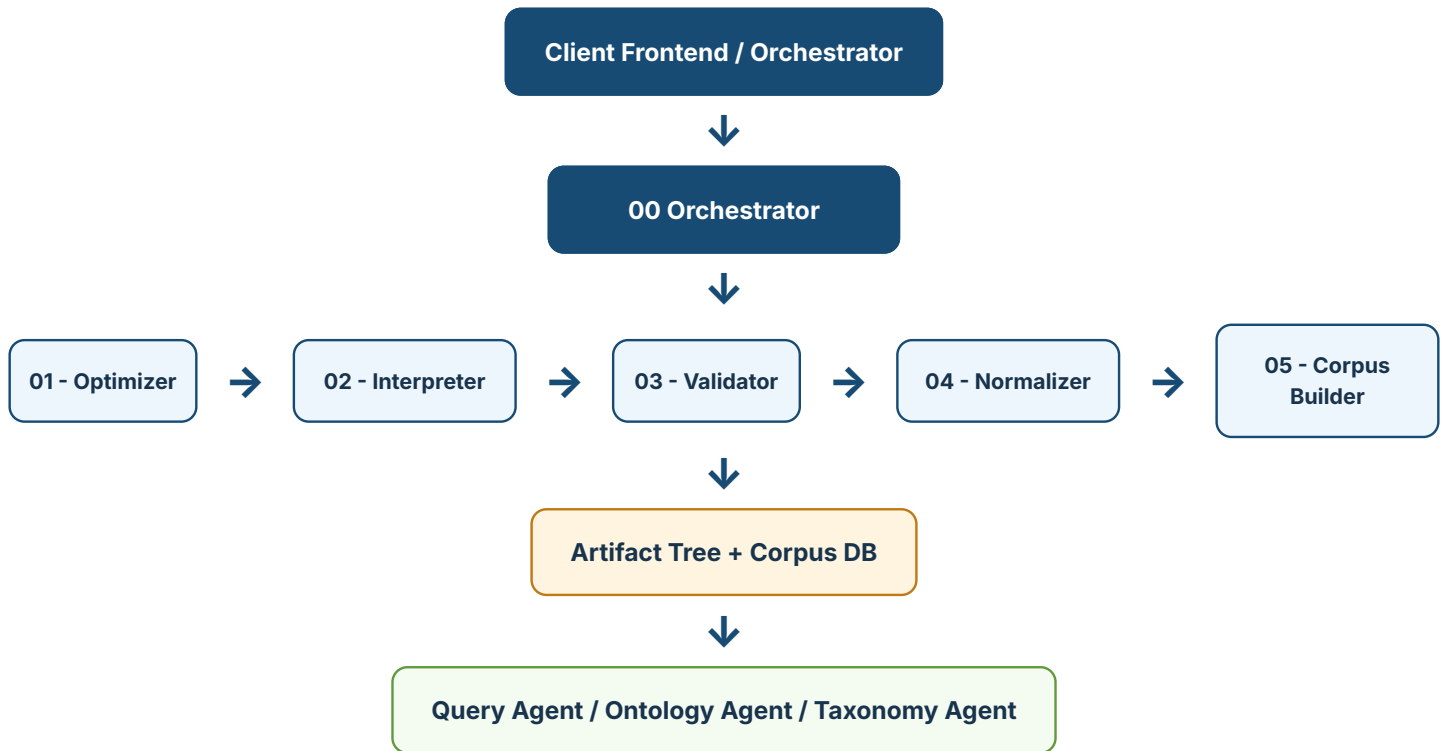


Figure 1. The short runtime path: orchestration, transformation modules, materialized corpus, and agent surfaces.

The long version is a set of connected owner surfaces. The Machine works because every surface has a limited job and the important runtime state is not quietly moved into the wrong place.

# Top-Level Topology

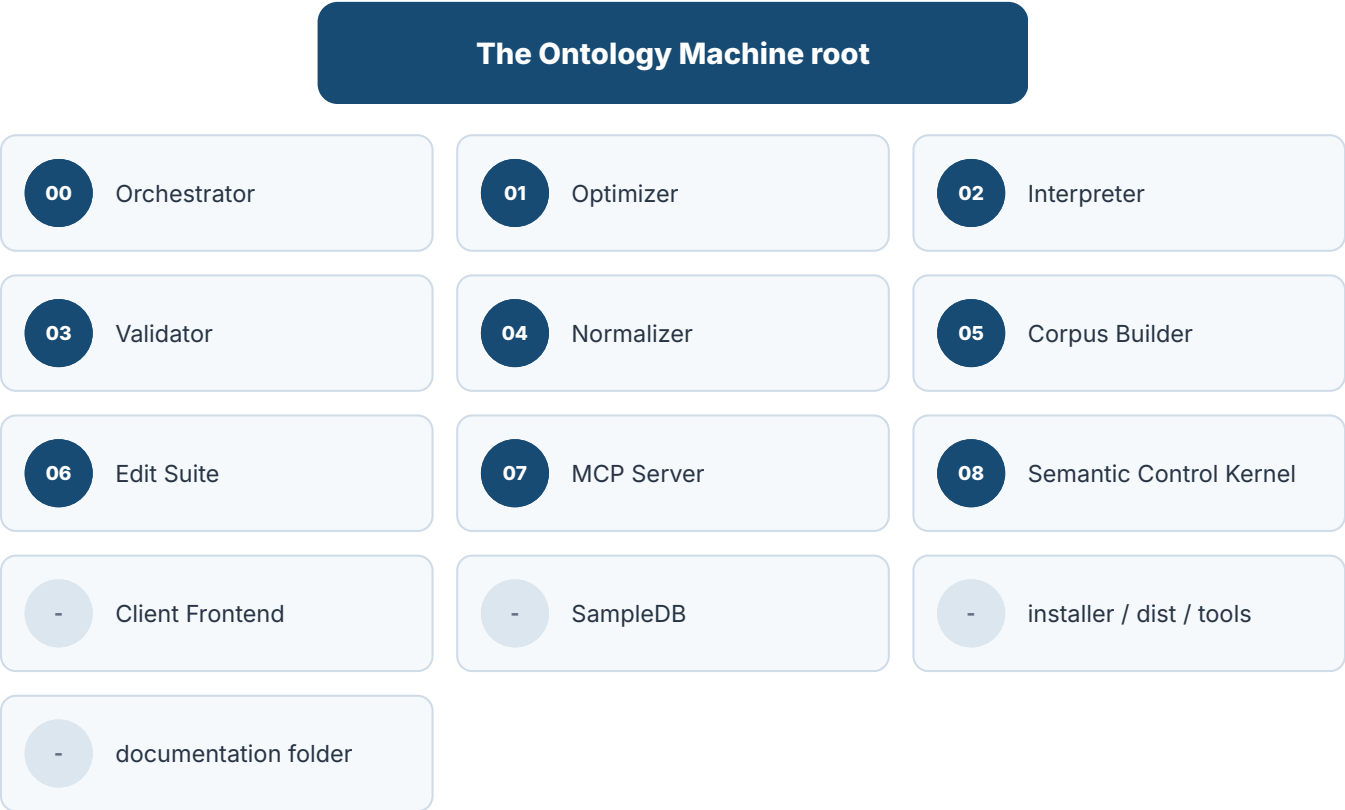


Figure 2. Top-level surfaces: the mainline modules plus surrounding product, transport, sample, and packaging surfaces.

The first six numbered modules form the ingestion and materialization mainline. The remaining surfaces are product control, user interaction, local tool transport, documentation, samples and packaging.

The Orchestrator module registry only lists the transformation modules:

| Registry alias | Module path            |
|----------------|------------------------|
| optimizer      | ../01 - Optimizer      |
| interpreter    | ../02 - Interpreter    |
| validator      | ../03 - Validator      |
| normalizer     | ../04 - Normalizer     |
| corpus_builder | ../05 - Corpus Builder |

Figure 3. The orchestrator registry lists only transformation modules. Other surfaces sit around the pipeline.

That means the Edit Suite, MCP Server, Semantic Control Kernel and Client Frontend are not hidden pipeline stages. They sit around the pipeline.

# Ownership Map

| Surface                      | Owns                                                                                                                         | Does Not Own                                                                   |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| 00 - Orchestrator            | pipeline runtime, stage scheduling, module discovery, artifact tree selection, runtime model/provider settings for ingestion | Corpus DB schema, taxonomy authoring, Kernel workflow state, frontend sessions |
| 01 - Optimizer               | source file intake, page assets, raw extraction payloads, OCR/extraction route                                               | document meaning, validation policy, normalized output, DB materialization     |
| 02 - Interpreter             | LLM interpretation of request-enriched page/document units                                                                   | validation decisions, canonical taxonomy mapping, DB writes                    |
| 03 - Validator               | validation reports, hard/soft failure signals, review flags                                                                  | release authoring, DB schema, ontology edits                                   |
| 04 - Normalizer              | taxonomy, projections, Semantic Release creation/validation, normalized payload contract                                     | Corpus DB schema, page images, pipeline scheduling                             |
| 05 - Corpus Builder          | SQLite schema, materialization, active release binding, embeddings, search, merge, rebuild, Base Graph, ontology schema      | model intent, UI dialogs, Kernel workflow state                                |
| 06 - Edit Suite              | advanced visible edit surfaces for owner-exposed config                                                                      | pipeline execution, foreign module direct writes                               |
| 07 - MCP Server              | local MCP transport, tool catalog, owner-contract delegation, Kernel bridge                                                  | business truth, workflow state, DB schema                                      |
| 08 - Semantic Control Kernel | workflow semantics, dialogs, progress, blockers, resume, recovery, receipts                                                  | UI rendering, MCP transport, DB schema primitives                              |
| Client Frontend              | browser UI, chat sessions, config UI, credentials UI, Query/Ontology/Taxonomy Agent surfaces                                 | owner module truth, Kernel state, MCP transport                                |

This table is the fastest way to orient a bug fix. If a symptom appears in the frontend but the wrong state was created by Corpus Builder, the fix belongs in Corpus Builder. If a workflow asks the wrong question, the fix belongs in the Kernel, not in MCP. If a document meaning is wrong before normalization, start with Interpreter and Validator artifacts, not the Query Agent.

# Mainline Ingestion Map

The Orchestrator execution policy names these stages:

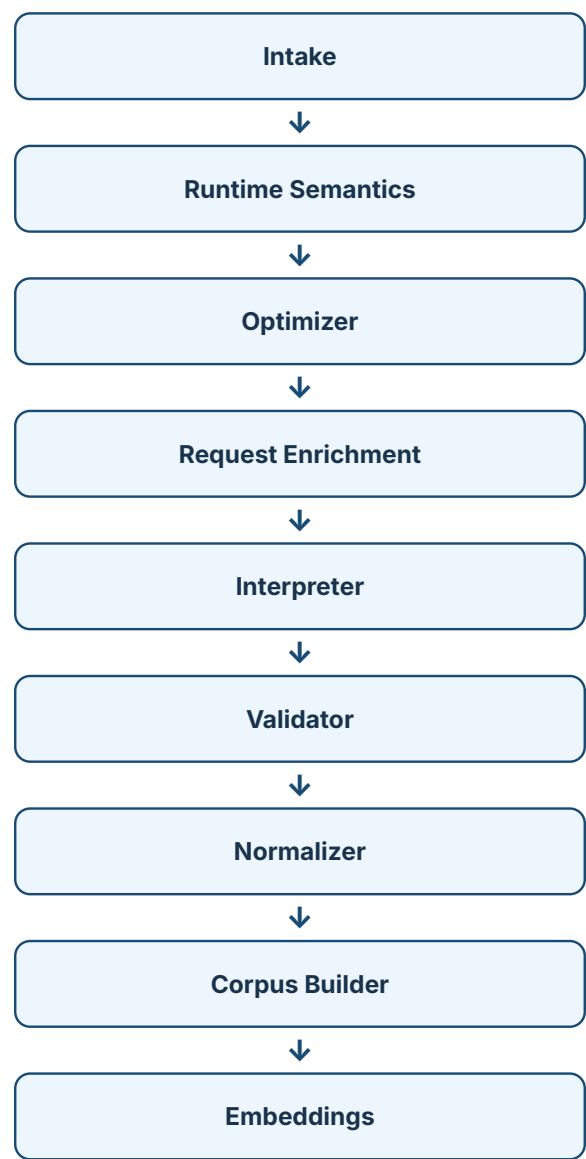


Figure 4. Orchestrator execution policy stages. Some stages are orchestration concerns rather than standalone modules.

Not every stage name is a separate module. `Intake`, `Runtime Semantics`, `Request Enrichment` and `Embeddings` are Orchestrator-side concerns around the registered module chain.

The registered runtime actions are:

| Module      | Required Actions                                  |
|-------------|---------------------------------------------------|
| Optimizer   | classify document, extract document, health check |
| Interpreter | interpret document, health check                  |
| Validator   | validate document, health check                   |

| Module         | Required Actions                                                                          |
|----------------|-------------------------------------------------------------------------------------------|
| Normalizer     | normalize document, build projection catalog, build runtime semantic assets, health check |
| Corpus Builder | load document, activate/read active Semantic Release, generate embeddings, health check   |

The practical ingestion flow is:

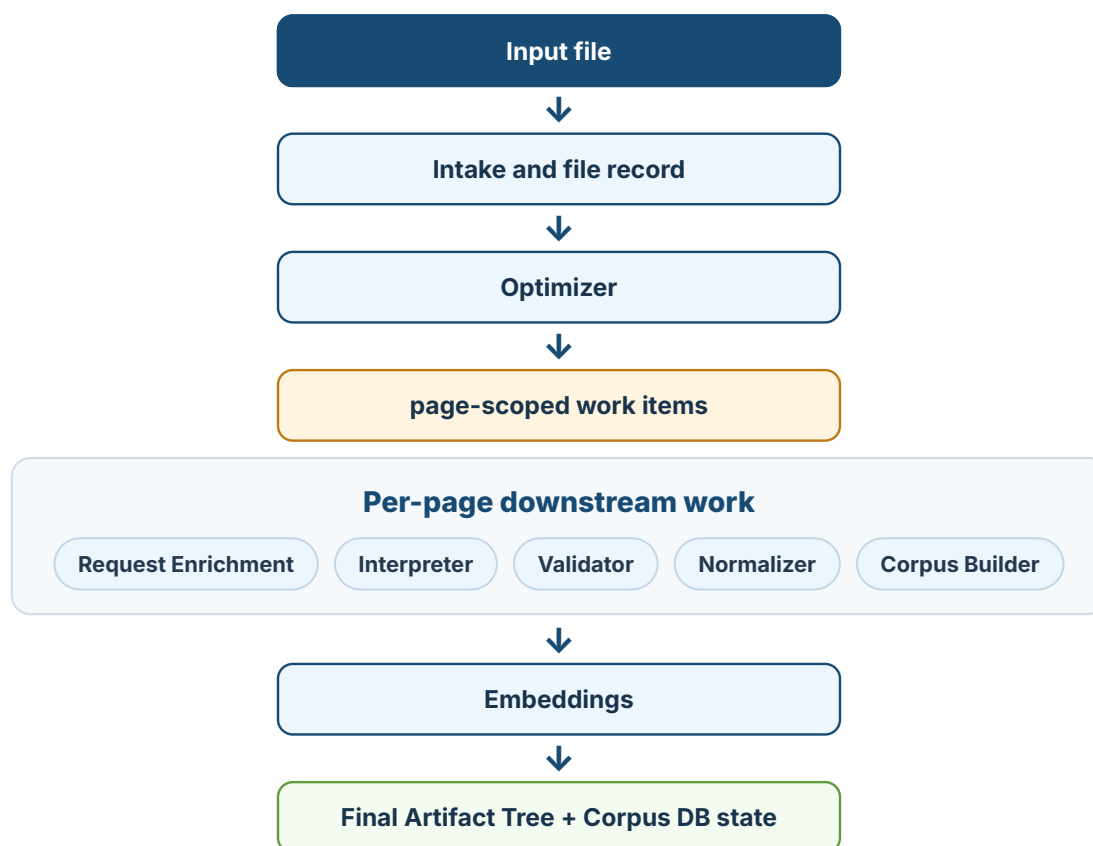


Figure 5. Practical ingestion flow. After optimization, work becomes page-scoped before final materialization and embeddings.

After Optimizer output exists, the active work becomes page-scoped. A multi-page document does not move through the downstream stages as one giant unit. It becomes ordered page work items, and the Base Graph later reconstructs source-document structure from the materialized identity fields.

This explains several runtime behaviors:

- progress can advance page by page
- error cases can belong to a single page of a larger source
- source documents must be reconstructed from page-level rows
- the Query Agent should use source-document readers for document-level answers
- page count and document count are not the same metric

# Artifact Flow

The Artifact Tree is the durable file surface around the run. The Corpus DB is the durable query surface after materialization.

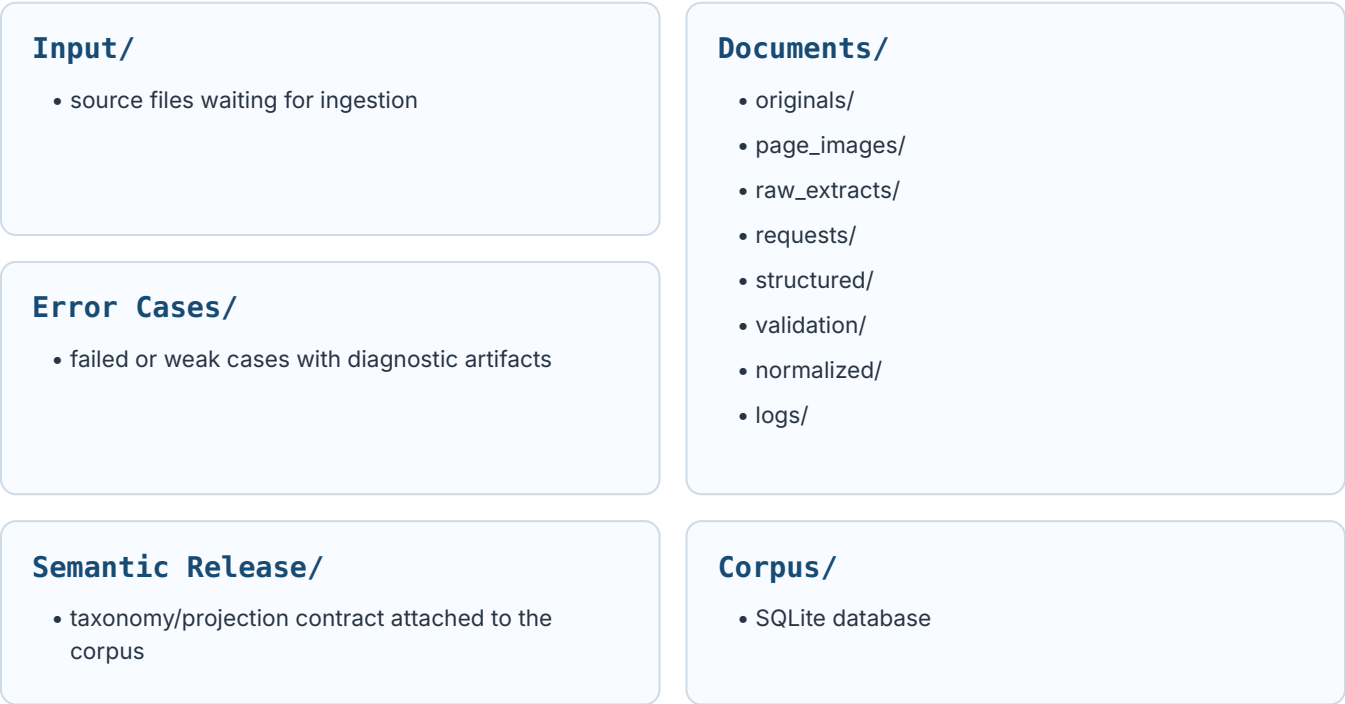


Figure 6. Durable file surface around a run: inputs, document artifacts, diagnostics, release package, and corpus database.

The important direction is:



Figure 7. The rebuild and audit surface feeds the durable query surface.

The DB can be queried by itself, but the Artifact Tree remains the rebuild and audit surface. The page images in Documents\page\_images are rebuild artifacts. The page images in document\_page\_images are DB-local evidence links.



# Corpus DB Layer Map

At a high level, the DB has these layers:

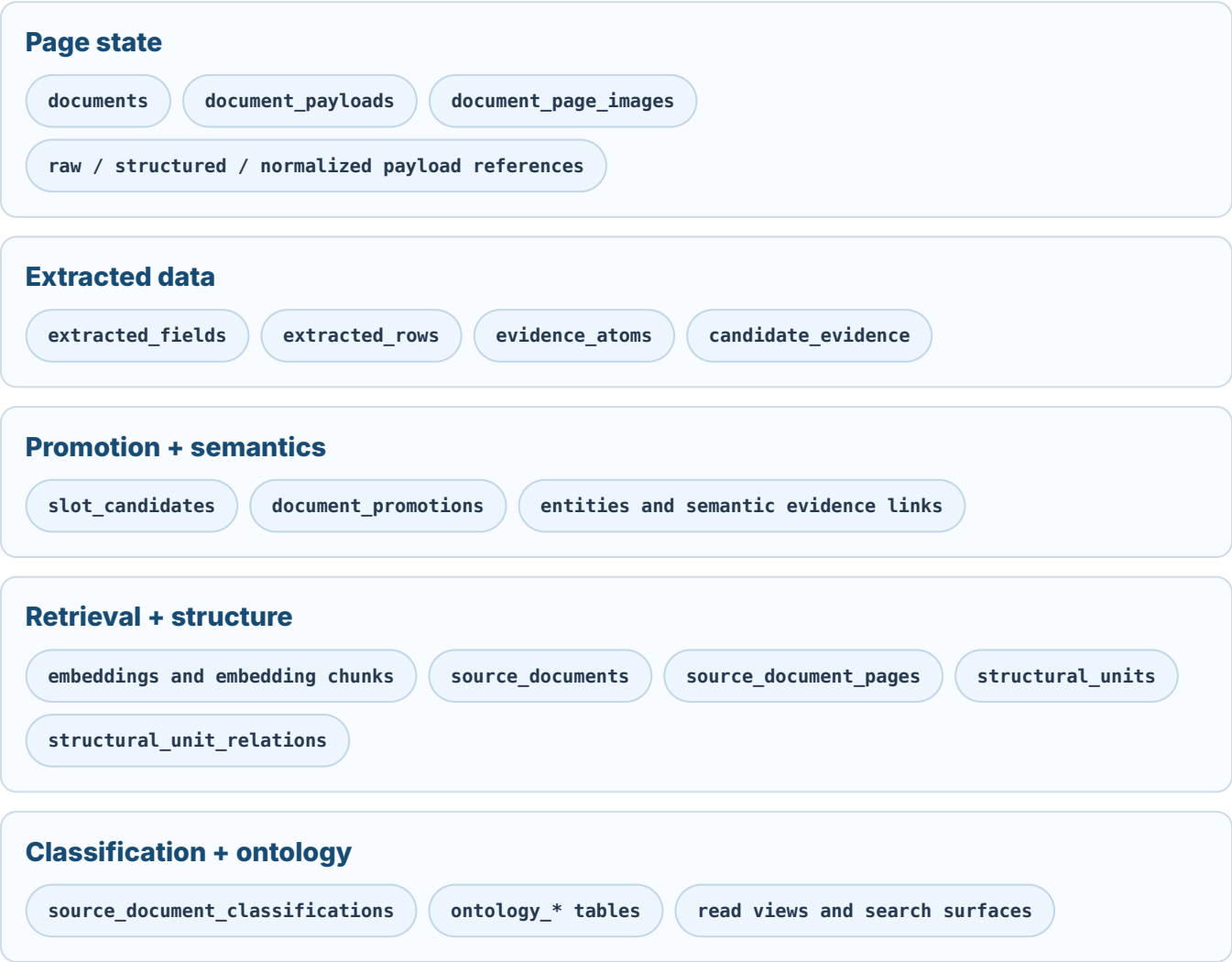


Figure 8. Corpus DB as a material stack: page state, extracted evidence, semantic slots, retrieval structures, and ontology lenses.

The DB layers have different roles:

| DB Area                     | Purpose                                                 |
|-----------------------------|---------------------------------------------------------|
| Documents and payloads      | page-level materialized document state                  |
| Page images                 | visible evidence surface inside the DB                  |
| Fields and rows             | normalized data extracted for query and comparison      |
| Evidence atoms              | small evidence units used for provenance                |
| Promotions and candidates   | normalized-first retrieval and important semantic slots |
| Entities and semantic links | structured entity surface and relation evidence         |

| DB Area           | Purpose                                                                  |
|-------------------|--------------------------------------------------------------------------|
| Embeddings        | semantic retrieval                                                       |
| Source documents  | deterministic grouping of pages into original source documents           |
| Structural units  | page/source/chapter/section-style structure                              |
| Classifications   | base, semantic release or ontology-scoped source-document classification |
| Ontology tables   | persistent lens-local knowledge graphs                                   |
| Read views/search | Query Agent and external reader surface                                  |

**Corpus rule:** The DB is not supposed to flatten all meaning into one table. It is a material stack that lets an agent move from broad search to exact evidence.

# Semantic Release Map

The Semantic Release is the bridge between model interpretation and DB materialization.

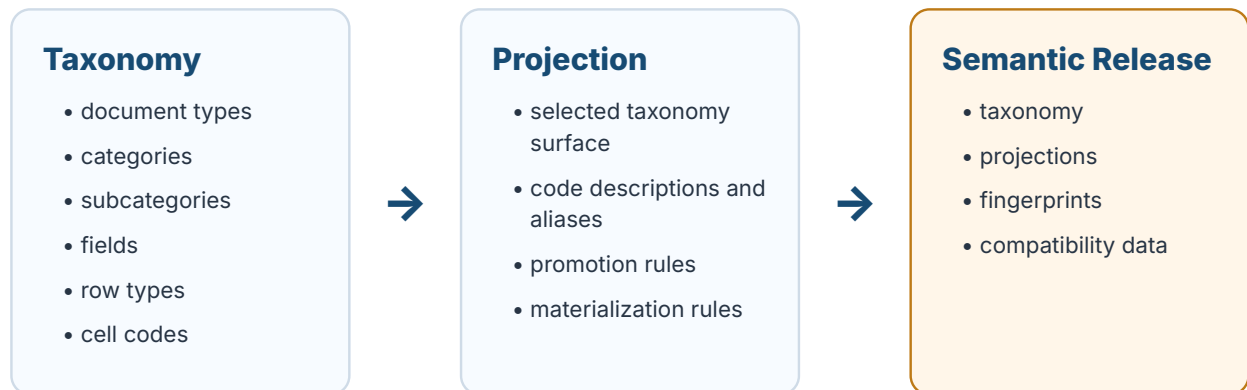


Figure 9. Semantic Release bridges model interpretation and DB materialization.

The Normalizer uses the active projection to produce canonical normalized payloads. The Corpus Builder uses the active release to materialize those payloads into the DB. Merge and rebuild workflows must respect the release fingerprint because materialized rows point back to release-defined IDs and codes.

# Control Plane Map

There are three control planes around the mainline:

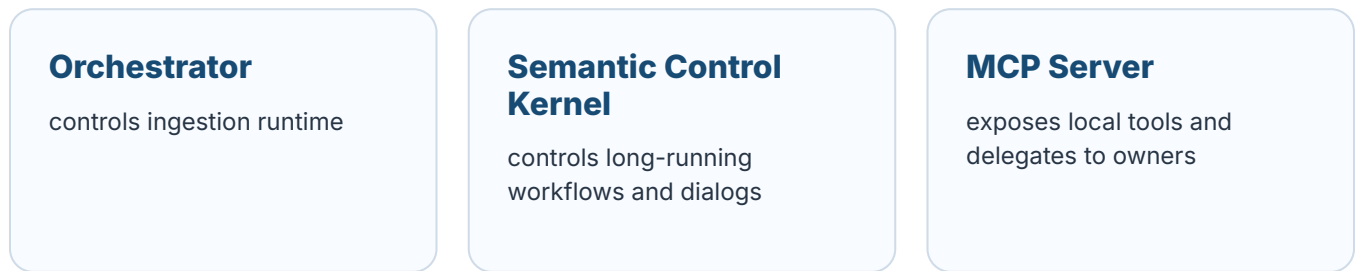


Figure 10. Three control planes wrap the ingestion mainline without taking over each other's truth.

## Orchestrator Control Plane

The Orchestrator runs the document mainline. It starts modules, enriches requests, schedules page work, tracks pipeline state, publishes artifacts and hands final success or failure back to the caller.

It is the correct starting point for:

- pipeline progress bugs
- startup and health check failures
- module action dispatch errors
- request enrichment issues
- artifact tree selection issues
- runtime provider/model injection for ingestion

## Kernel Control Plane

The Kernel runs long product workflows that would be too brittle if a chat agent tried to remember all steps in context.

It owns:

- workflow choice
- required user interactions
- confirmations
- progress and mirror events
- blockers and recovery classification
- resume options
- receipts and final notice context

It delegates actual domain work to owner modules. For example, a database merge is Kernel-coordinated but Corpus Builder-owned at the DB primitive layer.

## MCP Control Plane

The MCP Server is local stdio transport. It exposes tools, validates arguments and delegates to owners. It also bridges the Kernel so the Taxonomy Agent can see Kernel workflows as tools.

**MCP boundary:** The MCP Server should not become a second place where database truth, release truth or workflow truth is stored.

# Agent Map

The Client Frontend exposes three chat surfaces.



Figure 11. Client Frontend chat surfaces separate evidence answering, controlled ontology mutation, and Kernel workflow control.

## Query Agent

The Query Agent is the read-only corpus agent. It can use:

- read-only SQL
- document loading
- provenance tools
- semantic search
- coverage snapshots
- source-document readers
- ontology lens readers
- restricted read-only workbench access

Its job is to answer against evidence. The frontend also checks whether source links in the answer can be resolved against sources touched in the current tool turns. Unresolved sources are marked visually.

## Ontology Agent

The Ontology Agent can read the corpus like the Query Agent and can write only through controlled mutation tools:



`basic_relation_mining` is deterministic and builds the Base Graph from existing source identity fields.

`sql_batch_execute` writes ontology and allowed support tables through preflight checks, transactions, validation and logging. It is the safe version of agent write access: the agent can build persistent knowledge lenses without rewriting the base extraction and materialization path.

## Taxonomy Agent

The Taxonomy Agent is the user-facing workflow agent for the Kernel. In code, the older `pipeline_agent` name still appears, but the user-facing role is Taxonomy Agent.

It sees Kernel workflow tools through MCP. The permanent workflow tools cover:

- empty database creation variants
- custom taxonomy and custom projection paths
- manual pipeline run
- additive database merge
- rebuild from artifacts
- reset database
- Kernel status
- Kernel resume state
- continue resumable workflow
- cancel active run

The Taxonomy Agent should not manually invent paths, release state or resume payloads. Those values are Kernel/UI-owned.

# HTTP Surface Map

The Client Frontend server exposes the local browser app and the agent routes.

Core chat routes:

| Route                              | Surface                  |
|------------------------------------|--------------------------|
| POST /api/v2/chat                  | Query Agent              |
| POST /api/v2/pipeline-manager/chat | Taxonomy Agent code path |
| POST /api/v2/ontology-agent/chat   | Ontology Agent           |

History routes are split by surface:

## History routes by surface

- /api/chat/history
- /api/pipeline-manager/history
- /api/ontology-agent/history

The route name `pipeline-manager` remains for compatibility with the existing frontend API, even though the user-facing name is Taxonomy Agent.

Normal mode and config mode are separate local server modes. Normal chat mode uses the main frontend surface; config mode exposes the configuration surface for model settings, prompts, paths and policy.



# State Map

The system has several state locations. They should not be mixed.

| State                                     | Normal Owner                                |
|-------------------------------------------|---------------------------------------------|
| Source files waiting for ingestion        | Artifact Tree <code>Input</code>            |
| Successful document artifacts             | Artifact Tree <code>Documents</code>        |
| Failed document artifacts                 | Artifact Tree <code>Error Cases</code>      |
| Attached release package                  | Artifact Tree <code>Semantic Release</code> |
| Materialized corpus                       | <code>Corpus/*.db</code>                    |
| Pipeline run state                        | Orchestrator state                          |
| Kernel workflow state                     | Semantic Control Kernel state               |
| MCP transport/config state                | MCP Server state/config                     |
| Frontend sessions, config and credentials | Client Frontend state/config/vault          |
| Edit Suite cache/UI state                 | Edit Suite state                            |

**State rule:** Runtime state can describe or point at domain truth, but it should not silently become domain truth.

# Merge And Rebuild Map

Database merge and rebuild are important because they prove whether the system is really self-contained.

## Filled Database Merge

An additive filled merge must preserve:

- materialized documents
- page images
- payloads
- evidence
- promotions
- entities
- embeddings
- source-document structure
- Base Graph rows
- ontology lenses

It must not freely merge filled projections into a new projection truth. Filled rows already reference release-defined IDs, codes and fingerprints. If those contracts are rewritten after materialization, the DB may look merged while its semantic references are no longer trustworthy.

## Rebuild From Artifacts

Rebuild means the Corpus Builder recreates a DB from the Artifact Tree and its Semantic Release. This can recreate deterministic materialized state and the Base Graph can be run again afterward.

Ontology lenses are post-materialized knowledge. A plain artifact rebuild does not recreate them unless they were preserved or imported separately.

# Failure Routing Map

Use this table to find the first layer to inspect.

| Symptom                                              | Start Here                                                  |
|------------------------------------------------------|-------------------------------------------------------------|
| Source file will not open, render or split correctly | Optimizer and Orchestrator intake                           |
| Pipeline progress hangs or counts pages incorrectly  | Orchestrator scheduler and pipeline state                   |
| Interpreter output is semantically strange           | Interpreter request/response artifacts                      |
| Values disappear or drift before DB write            | Validator report and Normalizer payload                     |
| Document type/category is wrong                      | Normalizer projection and Semantic Release                  |
| DB has missing fields, rows or evidence              | Corpus Builder materialization                              |
| Multi-page source is treated as loose pages          | Base Graph/source-document tables                           |
| Query Agent misses documents                         | Query read surface, source-document tools, embeddings       |
| Query Agent cites suspicious sources                 | frontend source resolution and current-turn source list     |
| Ontology write fails with IDs, FK or NOT NULL errors | Ontology Agent preflight and Corpus Builder ontology schema |
| Taxonomy Agent workflow blocks or resumes badly      | Semantic Control Kernel state and workflow route            |
| MCP tool appears but fails execution                 | MCP bridge and owner contract call                          |
| Config page or credentials fail                      | Client Frontend config/credentials server routes            |
| Edit Suite surface is missing or stale               | owner edit contract and Edit Suite discovery/cache          |

# Change Orientation

When changing the system, start from the owner of the broken truth.

| Change Needed                                              | Start In                     |
|------------------------------------------------------------|------------------------------|
| Pipeline scheduling, final notices, stuck runs             | 00 - Orchestrator            |
| Rendering, raw extraction, OCR route                       | 01 - Optimizer               |
| Structured LLM interpretation                              | 02 - Interpreter             |
| Validation strictness or review flags                      | 03 - Validator               |
| Taxonomy, projections, Semantic Release logic              | 04 - Normalizer              |
| SQLite schema, materialization, embeddings, merge, rebuild | 05 - Corpus Builder          |
| Advanced edit UI for owner config                          | 06 - Edit Suite              |
| Tool transport and owner delegation                        | 07 - MCP Server              |
| Workflow state, dialogs, resume, blockers                  | 08 - Semantic Control Kernel |
| Query/Ontology/Taxonomy Agent UI and chat routes           | Client Frontend              |

**Debugging rule:** Do not patch the visible symptom just because it is visible. A frontend bug can be frontend-owned, but it can also be a bad Kernel event, a missing Corpus Builder row, a wrong Normalizer release, or an Interpreter artifact that already contained the error.

# Implementation Anchors

These are the main files to open when navigating the architecture:

| Area                     | Primary Anchors                                                                                    |
|--------------------------|----------------------------------------------------------------------------------------------------|
| Module registry          | 00 - Orchestrator/module-registry.json                                                             |
| Stage names/actions      | 00 - Orchestrator/config/execution_policy.json                                                     |
| Pipeline scheduler       | 00 - Orchestrator/orchestrator/pipeline/<br>stage_scheduler.py                                     |
| Module capabilities      | each module's module-manifest.json                                                                 |
| Corpus schema            | 05 - Corpus Builder/corpus_builder/database/                                                       |
| Base Graph               | 05 - Corpus Builder/corpus_builder/ontology/<br>basic_relation_mining.py                           |
| Merge/rebuild            | 05 - Corpus Builder/corpus_builder/<br>semantic_release/                                           |
| MCP stdio server         | 07 - MCP Server/mcp_server/server.py                                                               |
| MCP tool facade          | 07 - MCP Server/mcp_server/tools.py                                                                |
| Kernel tool surface      | 08 - Semantic Control Kernel/<br>semantic_control_kernel/surface/agent_tools.py                    |
| Kernel workflow dispatch | 08 - Semantic Control Kernel/<br>semantic_control_kernel/services/<br>agent_workflow_dispatcher.py |
| Frontend HTTP routes     | Client Frontend/client_frontend/http/                                                              |
| Query Agent              | Client Frontend/client_frontend/min_agent/                                                         |
| Ontology Agent           | Client Frontend/client_frontend/ontology_agent/                                                    |
| Taxonomy Agent code path | Client Frontend/client_frontend/pipeline_agent/                                                    |

# Reading Order

For a new maintainer, the architecture is easiest to understand in this order:

1. Read the System Overview.
2. Read this map until the owner surfaces are clear.
3. Open the module manifest for the area you want to change.
4. Open the owner contract or workflow entry point.
5. Only then inspect the frontend, MCP or agent surface that exposes it.

## The whole map in one line

The Machine becomes much less confusing once it is read as a set of owner surfaces around an evidence-bound corpus, not as one giant app with a chat window bolted onto it.